

Network Packet Analysis Report

HTTP, DNS and TCP Traffic Analysis Using Wireshark

Parameter Details

Parameter	Details
Prepared By	Rudresha RK
Internship Program	Cybersecurity Internship
Organization	InternSpark
Assessment Title	Network Packet Capture and Protocol Analysis Using Wireshark
Assessment Type	Network Traffic Analysis
Operating System	Kali Linux
Packet Analyzer	Wireshark 4.6.4
Network Interface	wlan0
Browser Used	Mozilla Firefox
Protocols Analyzed	DNS, HTTP/HTTPS, TCP, ICMPv6
Assessment Date	June 2026
Classification	Internal Use Only

1. Executive Summary

This assessment demonstrates practical packet capture and protocol analysis using Wireshark within a Kali Linux environment. The objective was to capture real-time network traffic, analyze commonly used networking protocols, inspect packet headers, and understand communication between network devices.

Traffic was generated by browsing websites, resolving domain names, and sending ICMP echo requests. Wireshark captured packets transmitted through the wireless interface (wlan0), allowing detailed inspection of DNS queries, HTTP/HTTPS communication, TCP acknowledgements, and ICMP packets.

Packet filtering techniques were used to isolate protocol-specific traffic. Individual packets were inspected to understand their headers, source and destination addresses, protocol fields, sequence numbers, acknowledgement numbers, and packet payloads.

The captured traffic successfully demonstrated normal network communication without identifying malicious packets. Some TCP retransmissions and duplicate acknowledgements were observed, which are common in real-world networks and do not necessarily indicate malicious activity.

2. Introduction

Network packet analysis is a fundamental skill in cybersecurity, digital forensics, and network administration. Every communication between computers is transmitted through packets that contain addressing information, protocol headers, and application data.

Packet analysis enables security professionals to investigate communication flows, troubleshoot network problems, identify suspicious behavior, detect malware communication, and understand protocol operation.

Wireshark is one of the most widely used network protocol analyzers. It supports hundreds of networking protocols and provides deep packet inspection capabilities for network troubleshooting and cybersecurity investigations.

During this assessment, live network traffic was captured and analyzed to study the behavior of DNS, HTTP/HTTPS, TCP, and ICMP protocols. Various display filters were applied to isolate protocol-specific traffic and inspect packet-level details.

3. Assessment Objectives

The objectives of this assessment were:

- Capture live network packets using Wireshark.
 - Analyze DNS requests and responses.
 - Inspect HTTP/HTTPS communication.
 - Study TCP packet structure.
 - Observe TCP acknowledgements.
 - Understand packet headers.
 - Apply protocol display filters.
 - Identify retransmissions and duplicate acknowledgements.
 - Document packet analysis findings.
-

4. Scope of Assessment

The assessment was performed entirely inside a controlled laboratory environment using Kali Linux.

The scope included:

- Live packet capture.
- DNS traffic analysis.
- HTTP/HTTPS packet analysis.
- TCP packet inspection.
- ICMPv6 packet analysis.
- TCP Analysis Flags inspection.
- Protocol filtering.
- Packet header analysis.

The assessment did not involve penetration testing or attacks against external systems.

5. Assessment Environment

Component	Configuration
Operating System	Kali Linux
Packet Capture Tool	Wireshark 4.6.4
Browser	Mozilla Firefox
Interface	wlan0
Network Type	Wireless Network
Protocols Captured	DNS, HTTP/HTTPS, TCP, ICMPv6
Display Filters	dns, http, tcp, tcp.analysis.flags

6. Tools Used

Wireshark

Wireshark is an open-source network protocol analyzer capable of capturing live traffic and performing deep inspection of hundreds of networking protocols. It allows analysts to inspect packets at every network layer, making it an essential tool for cybersecurity professionals, penetration testers, incident responders, and digital forensic investigators.

During this assessment, Wireshark was used to capture packets, inspect protocol headers, analyze TCP communication, review DNS queries, and monitor HTTP/HTTPS traffic.

Mozilla Firefox

Mozilla Firefox was used to generate network traffic by browsing websites. The browser generated DNS lookups, HTTPS communication, and TCP sessions, allowing protocol analysis within Wireshark.

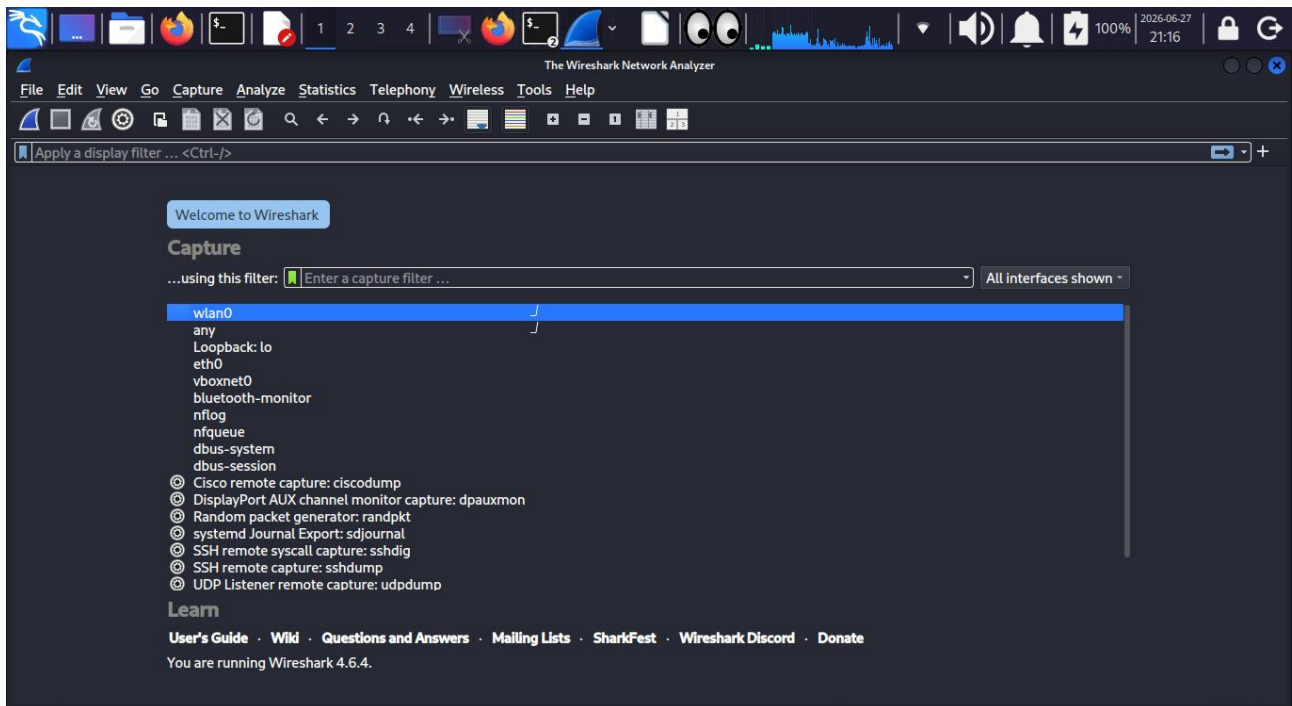
Linux Terminal

The terminal was used to generate ICMP traffic using ping commands to observe network connectivity and packet exchanges.

7. Practical Procedure

Step 1 – Launch Wireshark

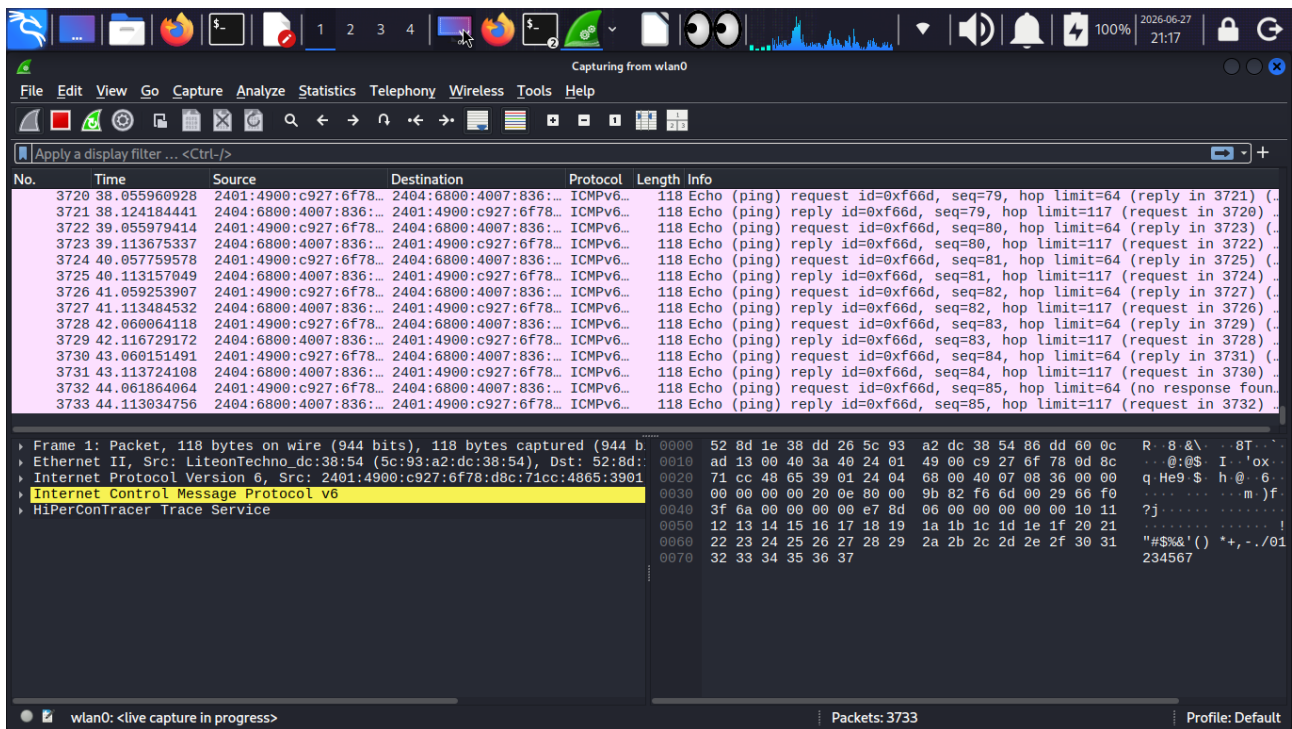
Wireshark was opened on Kali Linux and the available network interfaces were displayed.



Step 2 – Select Active Interface

The wireless interface (wlan0) was selected for packet capture because it was actively connected to the network.

Packet capture was started by clicking the blue shark fin icon.



Step 3 – Generate Network Traffic

Several activities were performed:

- Browse websites
- Resolve domain names
- Generate HTTPS traffic
- Ping remote hosts

This produced sufficient packets for analysis.

Step 4 – Analyze DNS Traffic

The display filter

dns

was applied.

DNS query packets and DNS response packets were displayed.

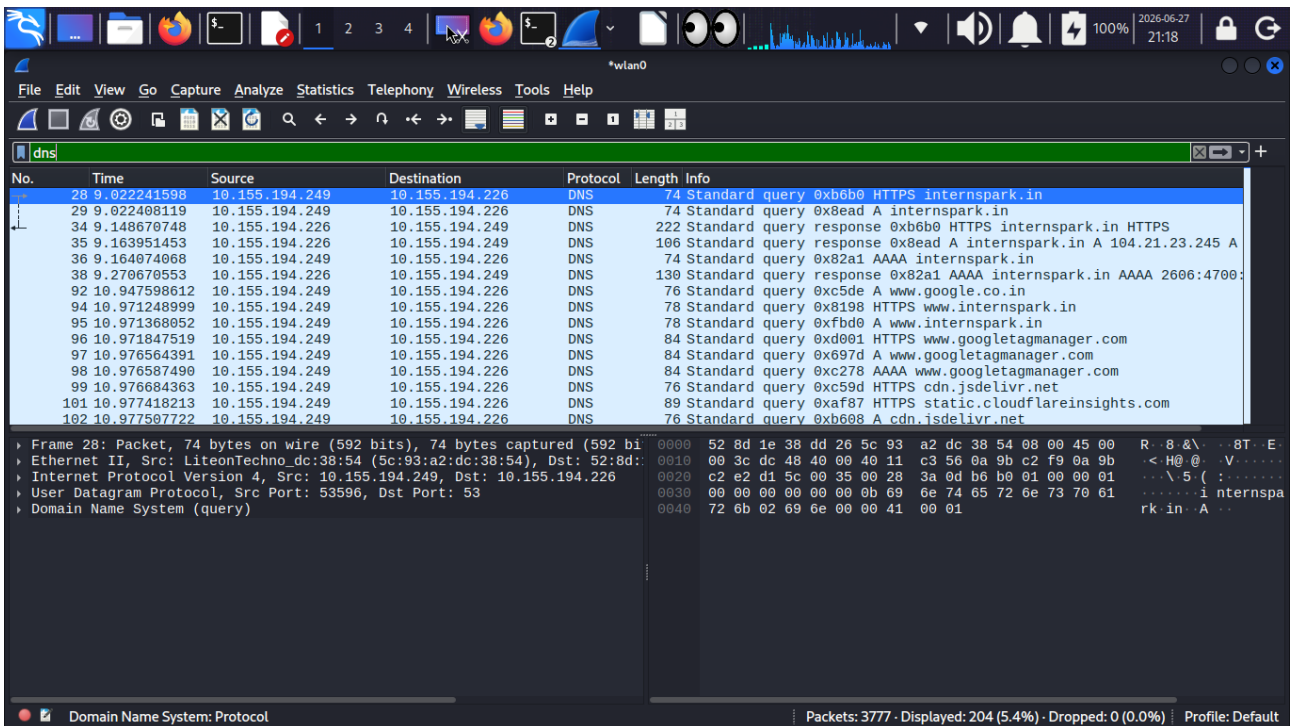
The captured packets showed hostname resolution requests for domains including:

- internspark.in
- www.google.co.in
- cdn.jsdelivr.net

- static.cloudflareinsights.com

Each DNS packet contained:

- Transaction ID
- Query Type
- Domain Name
- Response Records
- UDP Source Port
- Destination Port 53



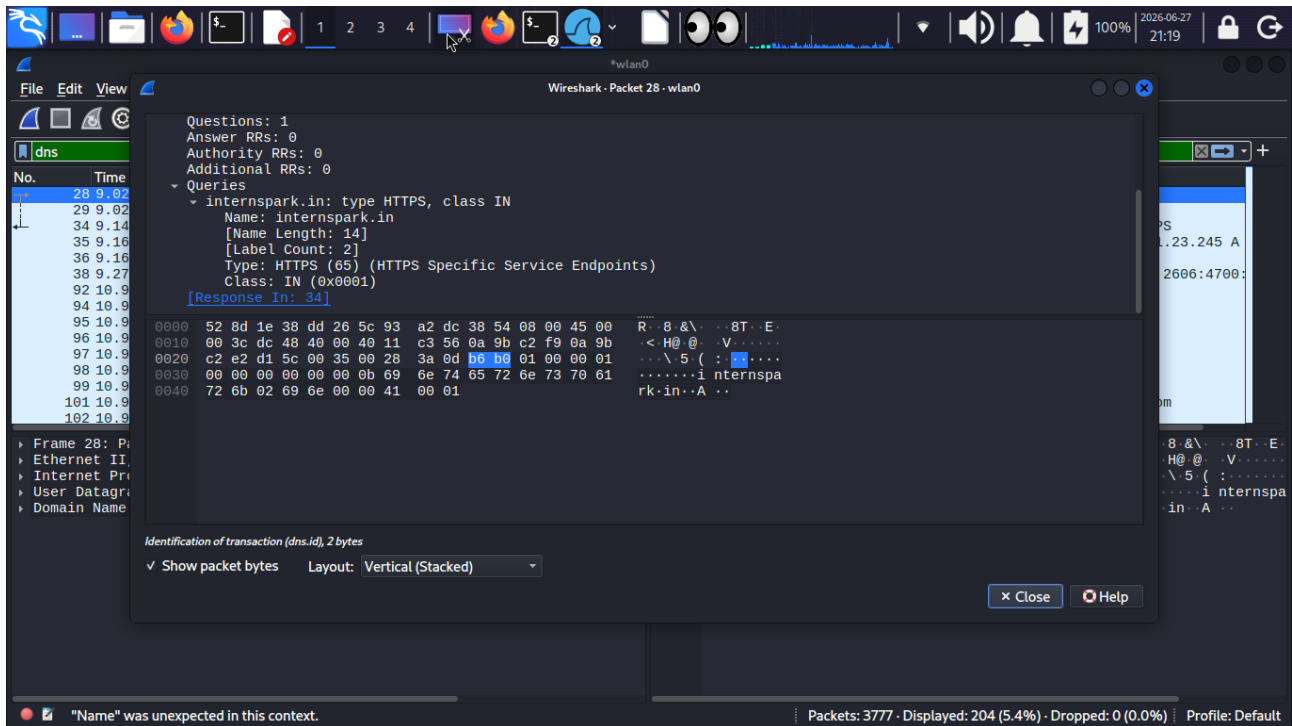
Step 5 – Inspect DNS Packet

One DNS query packet was selected.

Important packet fields observed:

- Transaction ID
- Questions
- Query Type
- Domain Name
- Class IN
- Response Frame

Packet bytes were also inspected.



Step 6 – Analyze HTTP/HTTPS Traffic

The display filter

http

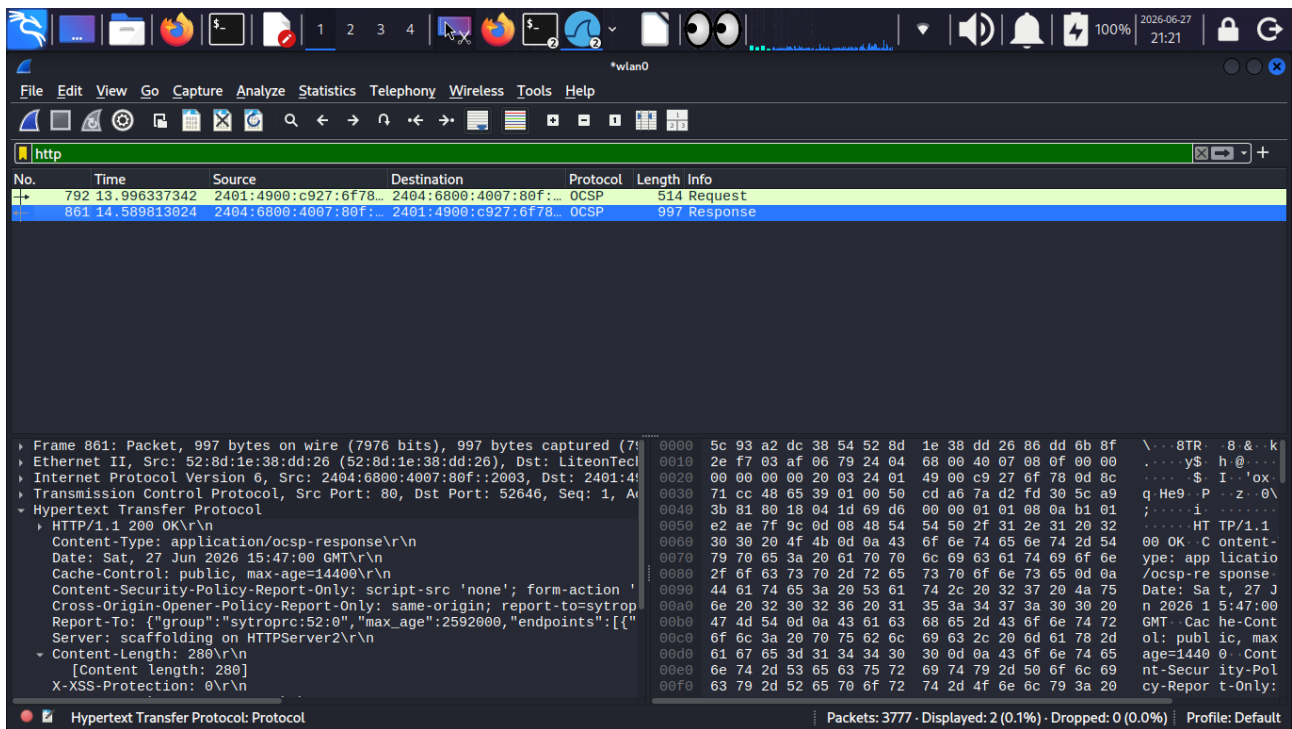
was applied.

The captured traffic included HTTP responses associated with OCSP communication.

Observed fields included:

- HTTP Version
- Status Code
- Content Type
- Content Length
- Cache Control
- Server Header
- Security Headers

The HTTP response status indicated successful communication.



Step 7 – Analyze TCP Packets

The filter

tcp

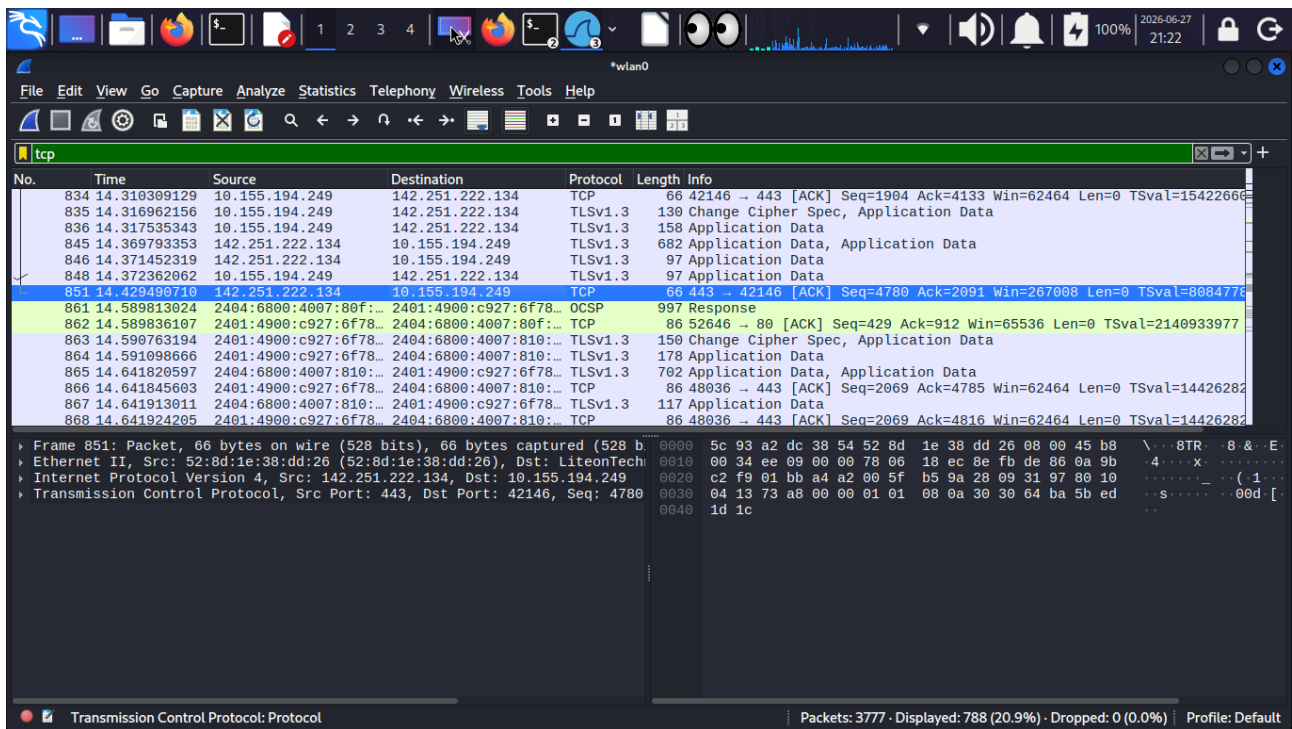
was applied.

Multiple TCP packets were captured showing communication between local and remote systems.

Packet details included:

- Source Port
- Destination Port
- Sequence Number
- Acknowledgement Number
- Window Size
- TCP Flags

The packet displayed represented a TCP acknowledgement packet.



Step 8 – Inspect TCP Header

A TCP packet was opened to inspect its detailed header.

The following fields were observed:

- Destination Port
- Stream Index
- TCP Flags
- Sequence Number
- ACK Number
- Conversation Completeness

The packet demonstrated successful TCP communication.



Step 9 – Analyze TCP Analysis Flags

The display filter

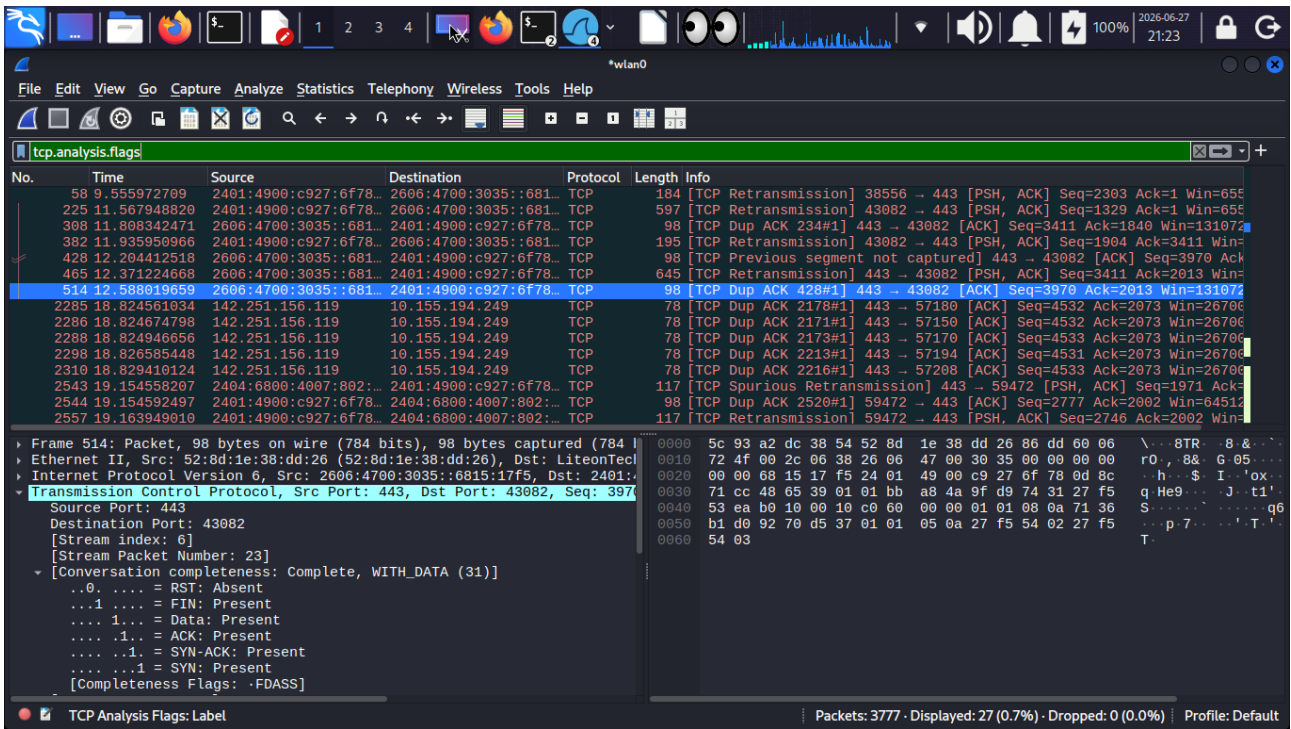
```
tcp.analysis.flags
```

was applied.

Wireshark identified several transport-layer events including:

- TCP Retransmission
- Duplicate ACK
- Previous Segment Not Captured

These events generally indicate temporary packet loss, delayed packets, or network congestion rather than malicious activity.



8. Protocol Analysis Summary

Protocol	Purpose	Observation
DNS	Resolves domain names	Queries and responses captured successfully
HTTP	Web communication	HTTP response headers inspected
TCP	Reliable transport	ACK packets observed
ICMPv6	Network connectivity	Echo requests and replies captured

9. Findings

The packet capture revealed legitimate network communication generated during web browsing and connectivity testing.

Key observations include:

- Successful DNS resolution.
- HTTP response packets captured.
- TCP acknowledgement packets observed.
- ICMPv6 echo requests and replies captured.
- TCP retransmissions detected.
- Duplicate acknowledgements observed.
- No malformed packets detected.

- No suspicious traffic identified.
-

10. Challenges Encountered

Several challenges were encountered during packet analysis:

- Modern websites primarily use HTTPS, limiting visibility into encrypted payloads.
 - HTTP filter displayed OCSP traffic instead of plain HTTP pages.
 - High packet volume required protocol filters for effective analysis.
 - TCP retransmissions required careful interpretation to avoid false positives.
-

11. Recommendations

The following best practices are recommended:

- Use protocol-specific filters during analysis.
 - Capture packets in isolated testing environments.
 - Analyze TCP flags to detect network issues.
 - Inspect DNS traffic for suspicious domains.
 - Combine Wireshark with IDS tools for advanced monitoring.
 - Maintain updated versions of packet analysis software.
-

12. Conclusion

This assessment provided practical experience in capturing and analyzing network traffic using Wireshark. DNS queries, HTTP responses, TCP acknowledgements, and ICMPv6 packets were successfully captured and inspected.

Protocol filtering enabled efficient analysis of individual communication types, while TCP Analysis Flags demonstrated how Wireshark detects retransmissions and duplicate acknowledgements. No malicious traffic or malformed packets were identified during the assessment.

The exercise strengthened understanding of network protocols, packet structures, and real-world traffic analysis techniques, providing valuable practical skills for cybersecurity and digital forensics.
